

INVESTMENT PORTFOLIO MONITORING AGENTIC WORKFLOW

1 Purpose of the project

The goal behind this project was to maximally automate monitoring of a personal thematic equity portfolio held in a broker account while keeping all the decision-making with the portfolio owner. Given that the portfolio contained over 20 positions stretched across several key themes, keeping up with developments in geopolitics, the broader economy, and individual industries would take a substantial amount of time and effort. The difficulty is compounded by relevance, since most business news concerns short-term developments and current dynamics that amount to mostly noise from the perspective of a portfolio anchored in multi-year theses built around long-term global trends. Reaching what actually matters therefore takes real work, and this is the main role of the agentic workflow, which filters hard for signal and compiles it into a comprehensive but easily digestible weekly briefing that forms the basis for the owner's decision-making. What it does not do is decide what the owner needs to see. The filtering is there to reach the signal, but once something has been identified as material, it is reported rather than gated on the workflow's own judgment.

Concretely, the system produces a single email every Saturday. It opens with the portfolio's headline numbers and a short synthesis of what mattered that week, then works through every position in turn, separating what actually happened from what the workflow makes of it. Each position also carries a check of its own trigger conditions, whether a price level laid out in advance has been crossed or a development in the news has met a condition the thesis was watching for. It closes with a new investment idea where one clears the bar, and any proposed updates to the research notes the system reasons from, each waiting on the investor's decision.

2 Design philosophy

First, given the thesis-driven nature of the investments, the workflow needs to focus on the datapoints that actually bear on the theses. This comes directly from the initial investment research, where every position was given a detailed investment thesis along with a valuation floor, named catalysts, kill switches, and staged entry triggers. Together these form the basis on which the workflow decides what counts as signal for a given position, since a piece of news matters mainly insofar as it moves one of these documented elements. To make this usable at runtime, the project uses a structured retrieval layer, a YAML file for every position holding its thesis, valuation, catalysts, deterministic and qualitative triggers, named risks, and a set of search hints, with the relevant file retrieved deterministically per position. This keeps every reasoning step grounded in the specific investment logic for that name rather than reasoning in the abstract.

Second, all judgement stays with the investor. The workflow can interpret data, lay out the case for an action, and advise, but it never executes on its own. In practice this shapes how the briefing is built. Each position is worked into a fixed structure that keeps the factual record separate from the interpretation, and the interpretive part is written to show its reasoning rather than just its conclusion, so the investor reads not only what the workflow thinks but why, and can disagree with it on the same terms. This audit trail carries most of the weight, because only price-based triggers are evaluated deterministically in Python, a threshold check being the one piece of logic

simple enough not to need judgement. Every other trigger, and the read on what a week's news and price action mean for a position, is left to the model. Since most of the reasoning genuinely sits with the LLM, the safeguard is making every conclusion legible and challengeable, not limiting how much the model is allowed to do.

Third, a knowledge base of this kind is only useful for as long as it stays accurate, and it drifts constantly as trades execute, catalysts fire, and company situations move. The workflow therefore helps maintain the files, but its writing privileges scale inversely with how much judgement a change requires. Purely mechanical changes, the kind with no real ambiguity, are synced automatically. Anything carrying a judgement call, a trigger status change for instance, is only proposed and waits for the investor to accept it. The underlying thesis prose is never rewritten by the workflow under any circumstance. This matters because these files are the reference the investor treats as ground truth, and a wrong automatic write would compound quietly until someone noticed. They can only change in ways either pre-authorised as mechanical or signed off directly.

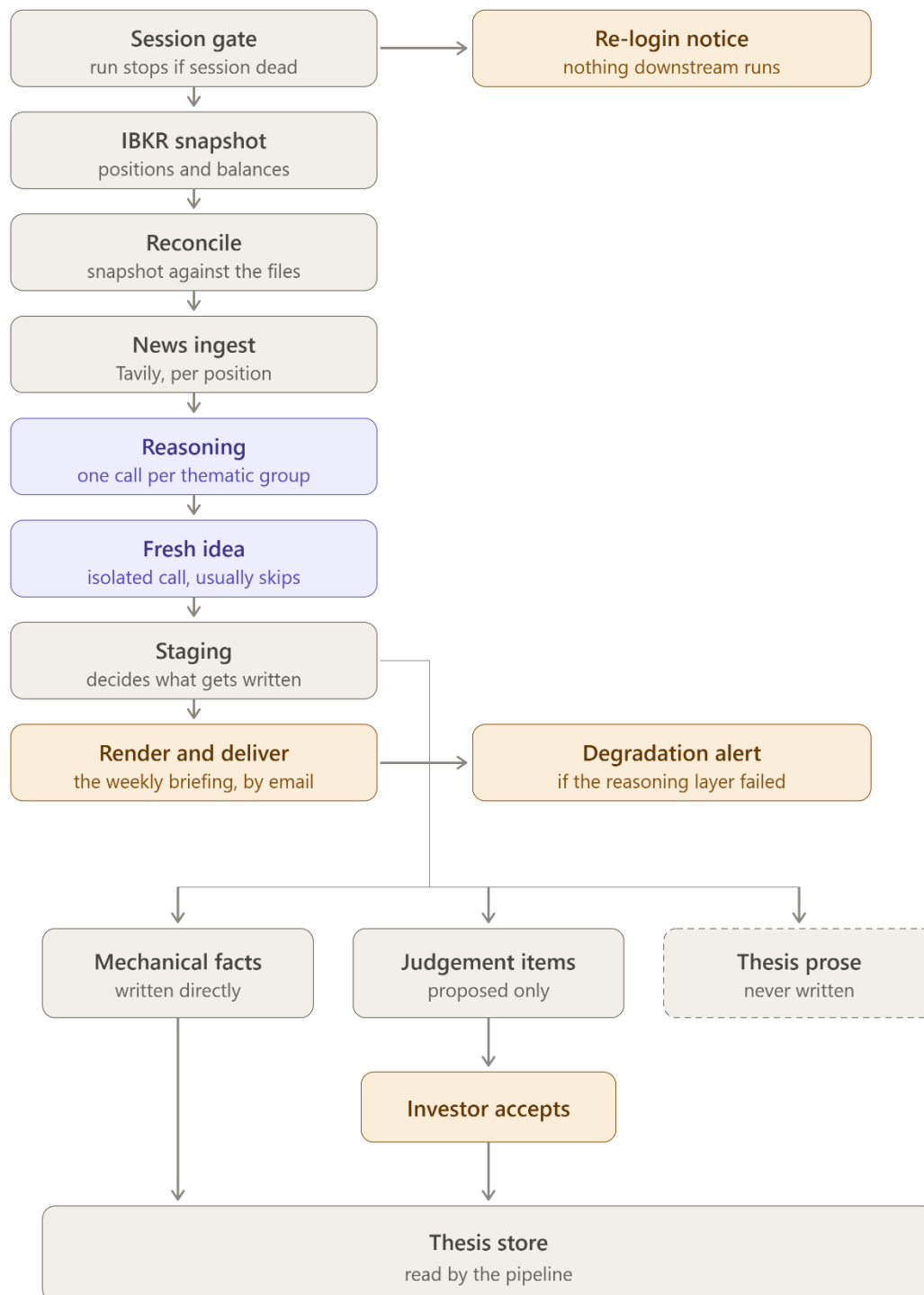
Fourth, the thesis files define what the workflow looks for, but they are not allowed to define what it is willing to report. Documented catalysts and named risks are by construction the things already thought of at the time the research was written, and the developments most worth knowing about are often precisely the ones nobody wrote down in advance. The briefing therefore carries a separate channel for anything material that maps to no documented catalyst, trigger, or risk, reported without being gated on whether it fits the existing frame. This is where filtering stops. Once something has been identified as important it is disclosed rather than quietly dropped for sitting outside the thesis, since a monitoring system that only ever confirms the shape of its own prior research is of limited use.

Fifth, the system should be free to extrapolate the investor's foundational research to generate novel ideas. This runs in two directions. Outward, the fresh idea generator proposes entirely new investment ideas, and inward, the workflow can propose a new trigger where a week's development reveals something the thesis has no forward-looking condition to catch. A fresh idea is held to the same standard as an existing position, meaning a specific instrument, a why-now argument grounded in a mispricing or a timing edge, a valuation floor, a named catalyst with a timeline, a bull-bear asymmetry of at least three to five times, a falsifiable disproof, and its relationship to the existing investments. A proposed trigger has to justify itself against what the current development implies about the future rather than against a pattern read off previous weeks. Neither is ever applied automatically, and both surface as proposals the investor either acts on or discards, so nothing enters the portfolio or the thesis files without an investor's decision.

Sixth, a quiet week and a failure both need to be disclosed transparently, but they are not the same thing, and the workflow is built to keep them distinct. If nothing material happened, saying so plainly is more useful and more honest than padding the briefing to look busy. But a genuinely quiet week and a partial failure can look identical from the outside, an empty section either way, and conflating the two is dangerous when the system runs unattended. So, a stage failure does not abort the run or shrink the briefing silently. The pipeline degrades gracefully and fills the gap with a fallback, but it also fires a separate alert naming the likely cause, and a dead brokerage session gets its own re-login notice rather than quietly producing an empty briefing. The effect is that an empty section can only ever mean nothing happened there, never that the step which should have filled it broke unnoticed.

3 System architecture

The diagram below sets out the weekly run end to end, from the session check through to delivery, together with the points where a person is involved and the three routes by which the run can change the underlying research files.



Purple: model judgement. Amber: what reaches the investor. Dashed: no write path exists.

Figure 1 - Weekly run overview

3.1 Key constraints

Majority of the system's shape is determined by how a retail account can reach its own brokerage data. Interactive Brokers exposes an OAuth path for programmatic access, but the self-service portal for it is institutional only and confirmed closed for this account, which leaves the Client Portal Gateway as the only workable route. The gateway is a Java process running on the local machine, serving a REST API over HTTPS on localhost, authenticated by a session the user opens once through a browser login. That session lapses roughly daily, so a weekly unattended run cannot assume it will find a live one waiting.

Two consequences follow. The first is that session state has to be handled as a normal operating condition rather than an error, so the run opens by pinging the gateway for portfolio data and, if the session is dead, sends a re-login notice through the delivery layer and exits before anything else executes. The practical target was almost full automation with about thirty seconds of human action to get the portfolio briefing, which is accepted as the design rather than tolerated as a shortfall. The second is that because the gateway binds to localhost, no cloud runner can reach it, which is why scheduling is a local Windows Task Scheduler entry wrapped in a PowerShell script rather than anything hosted. Both the manual login step and the decision to run locally therefore follow from the same access constraint rather than from any preference about how the system should be deployed.

3.2 Pipeline

The weekly run is a fixed linear sequence. The session check comes first, and nothing downstream executes unless it passes, since a briefing built on absent or stale holdings data is worse than no briefing at all. Past that gate the run pulls a full snapshot of the account from the gateway, covering current holdings, performance, and cash balances, and normalises it into a single consistent structure. Because the gateway reports each position in the currency of the exchange it trades on, values are converted to a common currency at this point so that positions can be compared against each other and against the portfolio as a whole.

The gateway session is the less predictable part, since it can respond slowly or return incomplete instrument data depending on how long it has been open. The client handles this with bounded retries on transient failures, a periodic keepalive so a longer run does not lose the session midway, and a secondary call that resolves each holding's full instrument name rather than leaving it as a bare ticker symbol, which matters because those names feed the news queries later in the run.

The snapshot is then joined against the position files to classify each holding as tracked, unfiled, or missing, and news is gathered for each position through the Tavily Search API, with queries built from that position's own search terms and near-term catalysts rather than from a single broad sweep. What gets filtered at this stage is deliberately mechanical only. Items older than two weeks are dropped, duplicates by URL removed, and a small denylist of low-quality domains applied. Whatever survives is capped at fifteen items per position, ranked by the relevance score the search provider returns with each result. That score reflects how closely an article matches its query, not how much it matters to the investment case, so the cap bounds volume rather than making any call about significance.

Reasoning then runs as a structured call to Claude Sonnet, once per thematic group of positions rather than once per position, which keeps related holdings in a shared context. The model is reached through a headless CLI, isolated behind a single module so that switching to a metered API later is a change in one place rather than throughout the codebase. Triggers expressed as a simple price threshold are evaluated in plain Python beforehand and passed to the model as already-decided facts, so it spends its judgement on the questions that actually need it. What comes back is the structured digest for each position. The staging layer compares it against the files to decide what gets written directly and what only gets proposed, before the result is rendered to HTML and sent.

Everything after the session gate degrades rather than stops. A failed search query for one position or a broken reasoning call for one group leaves a gap in that part of the briefing and lets the rest complete. The strictness at the front guards against acting on bad data, and the tolerance behind it against losing good data to an unrelated failure.

3.3 Layered model

Underneath the sequence the system separates into five layers whose boundaries are based on the design principles detailed above. The base is the thesis store, over 20 per-position YAML files holding thesis, valuation, staging status, catalysts, deterministic and qualitative triggers, named risks, and a live reference block of current figures. The pipeline reads these with PyYAML and writes them only through the staging layer, using a comment-preserving round-trip parser so an automated edit touches the intended fields and leaves formatting and annotations intact.

Above it, ingest draws on two independent sources, the gateway for holdings and performance and Tavily Search API for news, each normalised before anything interprets it. Keeping Tavily separate from the model rather than using Claude's own web search tool is deliberate, as will be discussed below. Reasoning is the only layer where interpretation happens. What it produces keeps the interpretive content in its own field, separate from the factual rows, and that separation is carried through the rest of the pipeline rather than being reconstructed at the point of rendering. Staging is the only write path back into the store, applying mechanical syncs directly, writing judgement-bearing items to per-position companion files for later acceptance through a small CLI, and never touching thesis prose. Delivery renders the digest to HTML and sends it through Resend, a transactional email provider. Every automated write to the position files is committed to git in the same run, which gives each week's changes a reviewable record and makes any of them straightforward to undo if the system got something wrong. Separating the layers this way means the trust boundaries hold by construction rather than depending on the reasoning step behaving well.

3.4 Integration choices

The brokerage client stays on Python's standard HTTP library rather than one of the available Interactive Brokers wrapper packages. The gateway's behaviour had already been validated against a live account before the wider system existed, and a wrapper would have re-abstracted the same quirks without removing any of them. The cost is that the normalisation logic is hand-maintained, so a gateway-side change becomes a problem to fix rather than someone else's upgrade.

News retrieval runs through Tavily rather than Claude's own web search tool, even though the latter would have removed a dependency and a stage from the pipeline. The difference is what

can be checked afterwards. When the search runs as its own step, the articles that reached the model are a concrete set that can be inspected and reproduced, so a questionable conclusion can be traced back to what it was working from. Had the model searched for itself, that set would exist only inside the call, and there would be no way to tell a weak read of good material from a confident read of material that was never there.

The model is invoked through Claude Code's headless CLI on a subscription plan rather than through the metered API. This avoids a separately billed key and a per-run invoice for what is a personal project. The invocation also sits behind a single module, so moving to the metered path later is a change in one file rather than across the codebase. The accepted risk is that a subscription login can require re-authentication with no browser present during an unattended run, which is handled the same way as the gateway session, by sending a notification rather than quietly producing a thinner briefing.

4 Build process

The order of construction followed one rule, that no layer was built until the one beneath it had been validated against real data. The first step was a standalone script with no AI component and no dependency on the rest of the system, doing nothing but authenticating against the gateway, pulling the core endpoints, normalising the result, and sending an email if the session was dead. It was run repeatedly against the live account, including a deliberate test of the failure notification, before anything else began.

Each layer after that was added the same way. The package was first scaffolded with every module present but unimplemented, so the structure could be reviewed before any of it worked. The brokerage client was then hardened, news retrieval built and tuned, and only then was the reasoning layer added, with its prompt tested against a single real week of data before being connected to the full run. The staging mechanism came last among the core steps, because it is the only part that writes back to the position files and there was no reason to give it access to them until everything upstream was known to work. Its YAML handling was checked against a copy of a real file until an automated edit changed exactly the intended fields and nothing else.

The most useful step was not a build step. Once the pipeline produced its first complete briefings, multiple rounds of review against real output led to prompt changes that would not have come up otherwise, covering how densely the reasoning was written and which figures it should avoid quoting. Neither issue was visible when reviewing the prompt on its own. Implementation throughout was done with Claude Code, and every automated write was committed to git from the first step, so the build history and the system's own audit trail are the same record.

5 Limitations

The reasoning layer has not been validated in any measurable way. Its output was judged by reading it across several real weeks, which is a reasonable standard for a personal tool but leaves no way to tell whether a trigger judgement was right in hindsight, or whether the model missed something a human reading the same articles would have caught. Missed items are the concerning case, since an omission is invisible in a way that a wrong call is not.

Nothing generated is tracked over time either. A proposed idea appears in that week's briefing and is then gone. This was deliberate, since a weekly suggestion should not accumulate into

something resembling a track record the system was never held to, but it does mean there is no evidence on how useful the generative layer actually is.

Operationally, the run is not fully unattended, it cannot be moved off the local machine without replacing the brokerage access path, and news retrieval sits on a free tier that the current cadence fits but would not survive much growth. More fundamentally, the design assumes a portfolio of this shape, concentrated, long-horizon, and documented position by position. Weekly cadence and reasoning against a written thesis both depend on that, and neither would transfer to a portfolio managed differently.

6 Concluding thoughts

The hardest part of this build was not making the system sophisticated, fast, or efficient, but making it genuinely useful to the person relying on it to make decisions. That turned out to be a much harder criterion to design against, because usefulness here is not a property of any single component and cannot be specified in advance. It came out of repeated iteration against real briefings, and most of what emerged was about restraint rather than capability, keeping retrieval separate from judgement so the evidence stays inspectable, confining automatic writes to mechanical facts and holding everything else as a proposal, distinguishing a quiet week from a failed run, and letting the generative layer decline rather than produce something to fill the slot. None of these make the system more capable and several make it less convenient, but they are what make its output something the investor can act on directly rather than having to verify first.